

Resenha — On the Criteria To Be Used in Decomposing Systems into Modules

Autor do Artigo: D. L. Parnas

Autor da Resenha: Felipe Luiz Parreiras Lima

Introdução

O artigo *On the Criteria To Be Used in Decomposing Systems into Modules*, de D. L. Parnas, é um dos textos mais relevantes da engenharia de software por discutir um problema central no desenvolvimento de sistemas: como dividir um sistema em módulos de forma realmente eficiente. Em vez de tratar modularização apenas como uma técnica de organização do código, o autor mostra que a qualidade dessa divisão depende diretamente do critério adotado no projeto.

Para sustentar sua argumentação, Parnas utiliza o exemplo de um sistema de geração de índice KWIC, comparando duas formas distintas de decomposição. A primeira segue a lógica tradicional, baseada nas etapas do processamento; a segunda propõe uma divisão orientada ao ocultamento de decisões de projeto. A partir dessa comparação, o artigo demonstra que a modularização não deve ser pensada apenas para fazer o sistema funcionar, mas para facilitar manutenção, evolução, compreensão e desenvolvimento independente.

Ideia central

A ideia central do artigo é que sistemas não devem ser divididos em módulos com base apenas no fluxo de execução ou nas etapas visíveis do processamento. Segundo Parnas, essa abordagem tradicional tende a expor decisões internas importantes entre vários módulos, o que dificulta alterações futuras e aumenta o acoplamento. Em vez disso, o autor defende que cada módulo deve esconder uma decisão de projeto que possa mudar com o tempo.

No exemplo apresentado, a primeira decomposição organiza o sistema em módulos como entrada, rotação circular, ordenação alfabética, saída e controle mestre. Essa estrutura parece intuitiva, mas faz com que vários módulos dependam diretamente de detalhes internos, como formatos de armazenamento, estrutura de tabelas e organização dos dados. Como consequência, uma simples mudança em armazenamento ou processamento impactaria diversos pontos do sistema.

Na segunda decomposição, o sistema é reorganizado em módulos como armazenamento de linhas, entrada, gerador de rotações, alfabetizador, saída e controle. Nessa proposta, cada módulo oferece uma interface mais abstrata e esconde sua implementação interna. Assim, detalhes como forma de armazenamento, estratégia de cálculo das rotações ou

momento da ordenação ficam encapsulados. Isso reduz a dependência entre partes do sistema e torna mudanças localizadas muito mais viáveis.

O artigo também introduz de forma clara o princípio de *information hiding* (ocultamento de informação), que se tornaria uma das bases da engenharia de software moderna. Para Parnas, o módulo bem projetado não é apenas aquele que executa uma função, mas aquele que protege o restante do sistema contra mudanças em decisões internas. Dessa forma, a modularização deixa de ser apenas uma divisão funcional e passa a ser uma estratégia de controle de complexidade.

Além disso, o autor destaca três benefícios esperados da modularização: redução do tempo de desenvolvimento por permitir trabalho mais independente entre equipes, maior flexibilidade para mudanças e melhor compreensão do sistema como um todo. Ele mostra que esses benefícios são alcançados com mais consistência quando a decomposição é feita com foco em decisões voláteis e não apenas em fases do processamento.

Aplicação em projetos reais

A contribuição do artigo é extremamente atual para projetos reais de engenharia de software, especialmente em sistemas corporativos, APIs, microsserviços, arquiteturas em camadas e aplicações escaláveis. Em ambientes modernos, é comum que equipes organizem módulos ou serviços com base apenas em fluxos do sistema, como cadastro, autenticação, pagamento ou relatório. Embora isso possa parecer adequado inicialmente, muitas vezes detalhes críticos de implementação acabam se espalhando por vários pontos da aplicação, gerando forte acoplamento e dificultando manutenção.

Na prática, a proposta de Parnas pode ser aplicada quando se define que certos módulos devem esconder decisões como persistência em banco, integração com APIs externas, mecanismos de autenticação, regras de cálculo, formatos de serialização ou estratégias de cache. Por exemplo, em um sistema corporativo, o módulo de autenticação deve esconder as particularidades de JWT, OAuth ou Cognito; o restante do sistema deveria depender apenas de contratos claros e não da implementação específica. Da mesma forma, uma camada de repositório deve encapsular detalhes do banco de dados, evitando que regras de negócio dependam diretamente da estrutura física dos dados.

Esse raciocínio também é muito útil em arquitetura de microsserviços. Um serviço bem definido não deve expor ao ecossistema interno suas decisões particulares de armazenamento, fila, ordenação ou processamento interno. Quando isso ocorre, torna-se possível trocar tecnologias, ajustar estratégias de performance ou reescrever partes internas sem quebrar outros serviços. Em termos de mercado, isso representa redução de custo de manutenção, maior adaptabilidade a mudanças de negócio e mais segurança para evolução contínua do produto.

Outro ponto relevante é a relação com boas práticas atuais como encapsulamento, baixo acoplamento, alta coesão, interfaces bem definidas e separação de responsabilidades. Mesmo sendo um artigo de 1972, o texto antecipa fundamentos que hoje aparecem em conceitos como Clean Architecture, Domain-Driven Design e design orientado a contratos.

Isso mostra que, apesar da evolução tecnológica, os problemas centrais de organização de software continuam os mesmos.

Conclusão

O artigo de D. L. Parnas é um marco na engenharia de software porque demonstra, com clareza e profundidade, que a forma de dividir um sistema influencia diretamente sua qualidade estrutural. Sua principal lição é que modularizar não significa apenas separar tarefas, mas identificar decisões importantes e voláteis do projeto e isolá-las dentro de módulos que escondam esses detalhes do restante do sistema.

A leitura é especialmente valiosa porque apresenta uma crítica objetiva à decomposição tradicional baseada em fluxogramas, propondo uma alternativa mais madura e mais adequada a sistemas complexos. O conceito de ocultamento de informação apresentado no texto continua extremamente atual e serve como base para decisões arquiteturais em projetos modernos.

Assim, o artigo permanece relevante tanto no contexto acadêmico quanto profissional. Seu principal aprendizado é que bons sistemas não dependem apenas de código funcional, mas de decisões de projeto que facilitem mudança, manutenção e compreensão ao longo do tempo. Para estudantes e profissionais de engenharia de software, trata-se de uma referência essencial para entender por que a arquitetura de um sistema deve ser pensada com foco em evolução e não apenas em execução imediata.